

Name :- Nimit jain

Reg.No :- 25BAI10840

Branch :- CSE(AIML)

Faculty :-Dr J. Sharmila Joseph

VIT BHOPAL
PROJECT REPORT

Title :- Banking-Management-System

Introduction:-

This project, the Banking Management System is a console-based application developed using the Java programming language. It is designed to simulate a fundamental core banking and account management system for a small branch, credit union, or financial unit. The core objective is to demonstrate the mastery of introductory and intermediate Computer Science principles, including modular programming, object-oriented design, fundamental data structures (Collections and Maps), and persistent data management through file input/output (I/O). A key feature of the application is the robust implementation of explicit input validation and business rule enforcement using control flow, ensuring data integrity without relying on external database software. The application allows users to open accounts, deposit funds, withdraw cash, perform inter-account transfers, and generate transaction statements, providing essential balance alerts and financial tracking.

Problem Statement

Manual banking record keeping and financial tracking (using spreadsheets or paper ledgers) is highly inefficient, prone to human calculation errors, and lacks the ability to provide real-time status updates. Errors in tracking balances and transactions can lead to critical discrepancies, unauthorized overdrafts, and operational losses.

Proposed Solution

The Banking Management System provides a simple, menu-driven interface accessible via the console. This solution automates the processes of recording banking transactions and utilizes a data file for persistence. This approach prioritizes simplicity, reliability, and clear demonstration of core programming logic over complex graphical interfaces or external database management systems.

Data Structure Design

The banking data is stored within Java as structured object models, leveraging the efficiency of the Java Collections Framework:

- Outer Structure (Accounts): A master Map where the Account Number (a string such as "SB-1001" or "CA-2001") acts as the unique key.
- Inner Structure (Details): The value associated with the key is an Account object containing encapsulated account details:
 - o 'accountHolderName': A string representing customer name.
 - o 'balance': A double tracking current monetary balance.
 - o 'hashedPin': A string storing salted SHA-256 password hash.
 - o 'transactions': A List storing chronological Transaction records.

Goal of this program:-

The main goal of the program is to provide an account management system for small banking units, cooperative branches, and clients to keep track of their balances and transactions. Using a simple menu-driven approach and direct accessing methods, this becomes an easy to understand program and thus is of practical real world use too. This system of management reduces the errors made by humans and keeps track of any account activity to fulfill requests immediately.

Explanation of the Program:-

This program begins by organizing domain logic across clean Java classes. The `java.io` package provides classes to interact with files on the filesystem in a portable way. Here we use object streams to interact with the data files of the operating system to store the state of accounts and access them as and when required by us. We then create several methods that contain the working of how accounts are accessed to get the balances or to store and update the values of transactions in them. The program is mainly built with the help of loops, if-else constructs, and basic console input/output statements to get the values. The program also verifies security PINs before releasing funds to prevent unauthorized access. The application is entirely menu-driven and implements the following essential banking features:

Data Persistence: Utilizes a standard file (`bank_data.dat`) to load and save all account data, ensuring that account information is retained between program executions.

executions.

Account Management: Allows users to open new accounts (Savings Account with initial deposit and interest rate, or Current Account with overdraft limit) and update balances via deposits, cash withdrawals, and inter-account transfers.

Input Validation: Employs explicit if/else logic to ensure that all inputs, particularly transaction amounts, are valid positive numbers, preventing logical errors.

Balance Restrictions & Alerts: Automatically checks if a savings balance falls below the minimum balance (\$100.00) or if a current account exceeds its overdraft limit, aiding in maintaining financial solvency.

Reporting: Displays account statements and transaction histories in a clear, formatted tabular view directly in the console.

Results

This application successfully manages the banking operations, providing a functional, persistent, and reliable text interface that accurately reflects real-world banking scenarios.

Conclusion:-

The Banking Management System project is a successful demonstration of core programming proficiency in Java. It effectively utilizes object-oriented principles for data organization, methods for modular design, and robust if/else conditional logic for comprehensive input and security validation. The successful implementation of data persistence confirms the student's understanding of File I/O and

fundamental application design.

What I learnt in this project:-

While making this project on banking management, I learnt the use of the java.io package and how it interacts with the Java program to store values so that they can be accessed anytime when needed. I gained the knowledge of how to store files in the directories of the system and can be used as an information management system also. I also learnt a broader use of if-else and looping constructs. While working out the program, I also got to know about try-catch type of constructs and got to know how it handles exceptions cleanly without stopping the program abruptly, allowing error detection quickly without excessive nested logic statements.

Future Scope

To evolve this project beyond the introductory level, the following enhancements could be implemented:

1. Database Migration: Replace file storage with a relational database (e.g., SQLite or MySQL) for better query performance and structured data management.
2. Graphical User Interface (GUI): Implement a desktop interface using libraries such as Swing or JavaFX to improve user experience.
3. Advanced Alerting: Integrate functions to check and alert the user about low balances or overdraft usage via email or SMS notifications.
4. Reporting: Add the ability to generate printable reports (e.g., PDF or CSV) of all recent transactions or bank liquidity audits.